[문제풀이] CronJob 생성하기

- 문제

kubernetes.io ▼

Keyword cronjob Job kubectl create job

TASK

Create a CronJob named **batch** that in **prod** namespace.

Container Spec:
- name: busybox
- image: busybox:stable
- command: ["sh", "-c", "date"]

Configure the CronJob:
- excute once every 30 minues
- keep 120 failed jobs
- keep 2 completed jobs
- 2 completions & 3 retries on failure
- never restart Pods
- terminate Pods after 30 seconds

Manually create and evecute one job named **batch-test** from the **batch** CronJob for testing

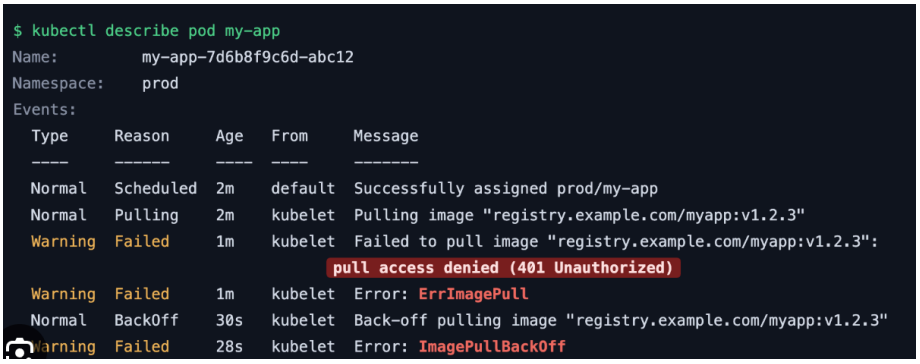
  - 주어진 spec으로 cronjob 생성
  - 생성한 cronjob을 사용하여, job을 직접 생성

- 정답
  - cronjob 생성

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: batch
  namespace: prod
spec:
  schedule: "*/30 * * * *"
  failedJobsHistoryLimit: 120
  successfulJobsHistoryLimit: 2


  # job의 template
  jobTemplate:
    spec:
      backoffLimit: 3 # 재시도 횟수 limit
      completions: 2
      activeDeadlineSeconds: 30



      # pod의 template
      template:
        spec:
          containers:
          - name: busybox
            image: busybox:stable
```

```
            command: ["sh", "-c", "date"]
          restartPolicy: Never
```

- 각 template의 구분
  - cronjob이나 deployment처럼 하위에 template이 여러 개 들어가는 경우, 어느 것의 option인지 tab으로만 구분하기 어려움
  - 위와 같이 각 template 사이에 공간을 두면, 구분하기 편하다.
- [”sh”, “-c”] 명령어
  - sh -c “xxxx”와 같이 명령어를 지정하면, shell을 사용하여 “xxxx”가 shell script처럼 해석해서 실행됨
  - 예를 들어 실행해야 하는 명령어가 while true; do date; sleep 10; done 와 같이 복잡한 명령이더라도, “sh”, “-c” 다음에 string 형태로 입력하면, 잘 동작한다.
- k8s에서는, 재시도 횟수를 지정할 때 필드명이 ‘retry’ 와 같이 되어있지 않고 ‘backoff’와 같이 되어있는 경우가 많다.
  - k8s에서는 실패 후 재시도할 때, 처음에는 10초후 시도, 그 다음에는 20초 후 시도... 와 같이 재시도 간격을 늘려나가는 경우가 많은데, ‘backoff’가 재시도와 재시도 간격에 관한 의미를 모두 포함한다고 한다.
  - pod가 뜨지 않아서 describe 해보면, 아래와 같이 ‘ImagePullBackOff’ 인 경우가 많은데, image를 pull 받는것에 실패해서, 계속해서 image pull을 재시도 하는 것이다. (재시도 간격은 점점 늘어남)

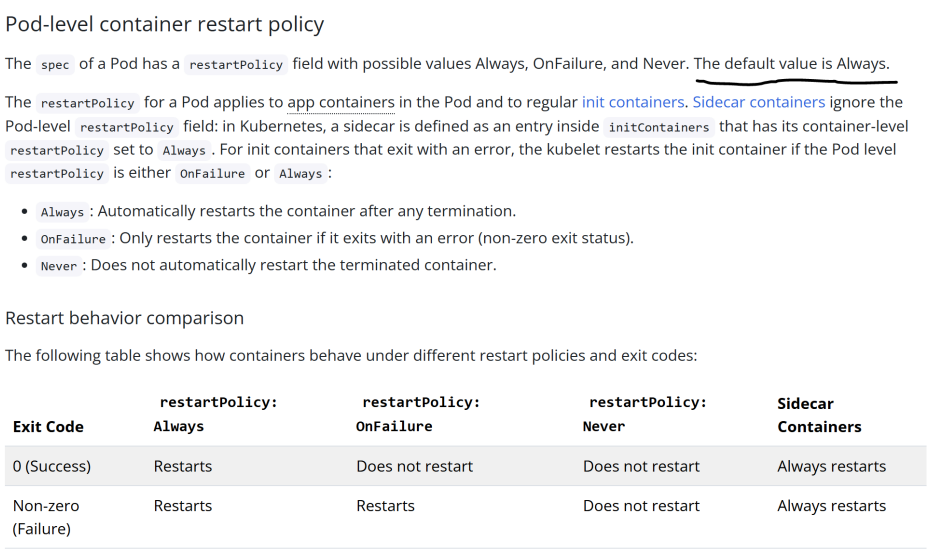


  - 이전 Mumshad의 강의에서 deployment가 제대로 동작하지 않는 이유를 확인하라는 연습문제가 간혹 있었는데, 위와 같이 ‘ImagePullBackOff’인 경우가 많았음.
- 실패 시 재시작하지 않도록 설정하는 ‘restartPolicy: Never’ 는 job의 spec이 아닌 pod의 spec이다.
  https://kubernetes.io/docs/concepts/workloads/controllers/job/#pod-template

> Pod Template #
>
> The `.spec.template` is the only required field of the `.spec`.
>
> The `.spec.template` is a pod template. It has exactly the same schema as a Pod, except it is nested and does not have an `apiVersion` or `kind`.
>
> In addition to required fields for a Pod, a pod template in a Job must specify appropriate labels (see pod selector) and an appropriate restart policy.
>
> Only a `RestartPolicy` equal to `Never` or `OnFailure` is allowed.

  - pod의 ‘restartPolicy’는 ‘Always’, ‘OnFailure’, ‘Never’ 등의 값을 선택 가능하다.
    (default는 ‘Always’이기 때문에, Job을 만들 때 pod의 template에는, ‘restartPolicy’ 값을 직접 넣어줘야 한다.)

https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-level-container-restart-policy



Pod-level container restart policy

The `spec` of a Pod has a `restartPolicy` field with possible values Always, OnFailure, and Never. The default value is Always.

The `restartPolicy` for a Pod applies to app containers in the Pod and to regular init containers. Sidecar containers ignore the Pod-level `restartPolicy` field: in Kubernetes, a sidecar is defined as an entry inside `initContainers` that has its container-level `restartPolicy` set to `Always`. For init containers that exit with an error, the kubelet restarts the init container if the Pod level `restartPolicy` is either `OnFailure` or `Always`:

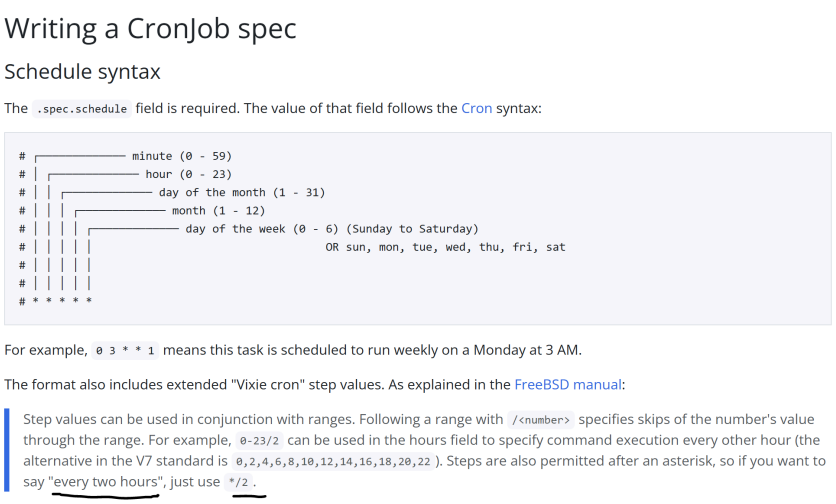
- `Always`: Automatically restarts the container after any termination.
- `OnFailure`: Only restarts the container if it exits with an error (non-zero exit status).
- `Never`: Does not automatically restart the terminated container.

Restart behavior comparison

The following table shows how containers behave under different restart policies and exit codes:

Exit Code	restartPolicy: Always	restartPolicy: OnFailure	restartPolicy: Never	Sidecar Containers
0 (Success)	Restarts	Does not restart	Does not restart	Always restarts
Non-zero (Failure)	Restarts	Restarts	Does not restart	Always restarts

- 매 30분마다 실행 (schedule: "*/30 * * * *")
  - cronjob 문서에, ‘every two hours’로 하려면 ‘*/2’로 설정하라고 하기 때문에, 매 30분마다 실행하고자 하는 경우, minute 부분에 ‘*/30’ 으로 설정한다.
    https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/#writing-a-cronjob-spec



Writing a CronJob spec

Schedule syntax

The `.spec.schedule` field is required. The value of that field follows the Cron syntax:

```
# ┌──────────── minute (0 - 59)
# │ ┌────────── hour (0 - 23)
# │ │ ┌──────── day of the month (1 - 31)
# │ │ │ ┌────── month (1 - 12)
# │ │ │ │ ┌──── day of the week (0 - 6) (Sunday to Saturday)
# │ │ │ │ │                                OR sun, mon, tue, wed, thu, fri, sat
# │ │ │ │ │
# │ │ │ │ │
# * * * * *
```

For example, `0 3 * * 1` means this task is scheduled to run weekly on a Monday at 3 AM.

The format also includes extended "Vixie cron" step values. As explained in the FreeBSD manual:

> Step values can be used in conjunction with ranges. Following a range with `/<number>` specifies skips of the number's value through the range. For example, `0-23/2` can be used in the hours field to specify command execution every other hour (the alternative in the V7 standard is `0,2,4,6,8,10,12,14,16,18,20,22`). Steps are also permitted after an asterisk, so if you want to say "every two hours", just use `*/2`.

- cronjob을 사용하여 job을 직접 생성

```
kubectl create job -n prod batch-test  --from=cronjob/batch
```

- k8s 문서에서 ‘kubectl create’ 로 검색해서 나온 페이지의, 아래 부분에 ‘See Also’가 있는데, 해당 부분에 가능한 모든 ‘kubectl create xxxx’ 명령어들 목록이 있다.
  https://kubernetes.io/docs/reference/kubectl/generated/kubectl_create/#see-also

See Also

- kubectl - kubectl controls the Kubernetes cluster manager
- kubectl create clusterrole - Create a cluster role
- kubectl create clusterrolebinding - Create a cluster role binding for a particular cluster role
- kubectl create configmap - Create a config map from a local file, directory or literal value
- kubectl create cronjob - Create a cron job with the specified name
- kubectl create deployment - Create a deployment with the specified name
- kubectl create ingress - Create an ingress with the specified name
- kubectl create job - Create a job with the specified name
- kubectl create namespace - Create a namespace with the specified name
- kubectl create poddisruptionbudget - Create a pod disruption budget with the specified name
- kubectl create priorityclass - Create a priority class with the specified name
- kubectl create quota - Create a quota with the specified name
- kubectl create role - Create a role with single rule
- kubectl create rolebinding - Create a role binding for a particular role or cluster role
- kubectl create secret - Create a secret using a specified subcommand
- kubectl create service - Create a service using a specified subcommand
- kubectl create serviceaccount - Create a service account with the specified name
- kubectl create token - Request a service account token

- 터미널에서 'kubectl create job —help' 와 같이 입력하면, k8s의 'kubectl create job' 페이지에 나와 있는 설명과 거의 동일한 것을 얻을 수 있다. (페이지의 'Parent Options Inherited' 부분과 'See Also'를 제외한 것을 얻을 수 있음)
  https://kubernetes.io/docs/reference/kubectl/generated/kubectl_create/kubectl_create_job/

```
[candidate@k8s-master ~]$ kubectl create job --help
Create a job with the specified name.

Examples:
  # Create a job
  kubectl create job my-job --image=busybox

  # Create a job with a command
  kubectl create job my-job --image=busybox -- date

  # Create a job from a cron job named "a-cronjob"
  kubectl create job test-job --from=cronjob/a-cronjob

Options:
    --allow-missing-template-keys=true:
        If true, ignore any errors in templates when a field or map key is missing in the template. Only applies to
        golang and jsonpath output formats.

    --dry-run='none':
        Must be "none", "server", or "client". If client strategy, only print the object that would be sent, without
        sending it. If server strategy, submit server-side request without persisting the resource.

    --field-manager='kubectl-create':
        Name of the manager used to track field ownership.

    --from='':
        The name of the resource to create a Job from (only CronJob is supported).
```

- cronjob에 "*/30 * * * *" 와 같이 설정하면, 10:30, 11:00, ... 과 같이 실제 시간 기준으로 30분마다 실행됨.
  - 실제 30분이 될 때까지 기다리지 않고, kubectl create job 명령어로 job을 수동 생성하여, job이 정상적으로 도는 지 확인할 수 있다.